
MCU Driver Software Development Kit Integration Guide

Introduction to MCU Driver SDK

The MCU Driver SDK is a freely available repository of Cirrus Logic driver code and tools for integrating certain Cirrus Logic CODECS/Amplifier devices into an MCU-based development system.

The SDK is available at <https://github.com/CirrusLogic/mcu-drivers>.

The MCU Driver SDK's intention is to provide a starting point and guidance to help integrate Cirrus Logic devices into an end system's host MCU software implementation.

The SDK's main components consists of example MCU device driver code, useful tools to help with development and device configuration, and platform BSP code to support the MCU platform.

Driver code shows how to perform the initialize, reset, boot, and configure stages. Use-case code attempts to perform the commonly useful features the particular device offers.

A set of software tools and scripts are provided to help with the integration stage and with configuring the device. The SDK has a layer of BSP code specifically for the STM32® Nucleo-64 F401RE MCU board which is the MCU used by Cirrus Logic for driver development.

This document provides reference to existing documentation and application notes where possible. If an existing document exists for a particular device, it should be the main point of reference for details on that device, together with the device's silicon datasheet.

A quick start section is provided which aims to get the reader up and running without the need for detailed explanations.

This document is part of a set of Application Notes, which provide greater detail for a particular device or feature. References to these Application Notes are used where appropriate. Links to references are given in [Existing Material Links](#).

Important Notice:

No license to any intellectual property right is included with this component, and certain uses or product designs, including certain haptics-related uses or haptics-system designs, may require an intellectual property license from one or more third parties.

Table of Contents

Introduction to MCU Driver SDK	1
1 Quick Start to Build Code	3
1.1 Prerequisites For Building	4
1.1.1 Code Layout Guide	4
2 Integration Guide Overview	5
2.1 Driver to BSP Interface	5
2.2 Device Configuration Data, Device Driver Instance	5
2.3 Baremetal and FreeRTOS Example Projects	5
2.4 Firmware Files	6
2.5 WISCE Script Configuration	6
3 Overview of Cirrus Logic MCU Driver Development System	6
3.1 Detailed System Architecture	7
4 Overview of Use-Case Code	7
5 Overview of Driver Code	9
6 Important Data Structures	9
6.1 regmap_cp_config_t	10
6.2 <device>_bsp_config_t	10
6.3 <device>_config_t	10
6.4 <device>_t	11
6.5 bsp_driver_if_t	11
7 Overview of BSP Code	11
8 Device Configuration Tools	12
8.1 WISCE Script Converter	12
8.2 Firmware Converter	13
8.3 MCU Driver Bridge	13
9 Existing Material Links	14
10 Revision History	14

1 Quick Start to Build Code

This section is a quick start to building the code directly from the GitHub repository as-is, and assumes you are using the same Cirrus Logic development system using the Lochnagar2 (LN2) development board connected to the STM32 Nucleo MCU board as shown in [High Level Architectural Diagram](#). To build working code for any other platform you will need to perform integration steps as shown in [Integration Guide Overview](#).

Refer to [Prerequisites For Building](#) to understand how to create a host build environment.

1. Ensure that the correct firmware file(s) are present, since the example MCU driver code is dependent on them.

Contact your Cirrus Logic representative to obtain the correct firmware files for your device. Some devices can operate in ROM mode without firmware. To verify, search for instance of the extensions ".wmfw" and ".bin" in the makefile under the *device* folder. Each firmware file needs to be placed in the *fw* folder inside the *device* folder. For example, *cs40l50/fw/CS40L50_Rev3.4.5.wmfw*.

The source tree contains a reference to the STM32CubeF4 GitHub repository which contains STM32 HAL, BSP, libraries and other support software provided by STMicroelectronics for the STM32 boards. This is contained as a git sub-module in the MCU Driver tree that needs to be pulled down only once per release.

```
$ git clone git@github.com:CirrusLogic/mcu-drivers.git
```

```
$ git submodule update --init --recursive
```

The git "master" branch should be used to build and will always point to the latest release.

2. Navigate to the appropriate device folder (root/<device>) in your shell and build either the baremetal or FreeRTOS® versions of the binary executable as shown below.

```
$ cd cs40l50
```

```
$ make clean; make baremetal -j8 # Builds the baremetal version of the binary
```

```
$ make clean; make freertos -j8 # Builds the FreeRTOS version of the binary
```

The build step will automatically integrate a firmware file's data into the binary if the correctly named firmware for the device is available in the <device>/fw folder. If no firmware is provided (or incorrectly named) the build will fail with output as shown below. This step uses the firmware_converter tool which is described in detail in [Device Configuration Tools](#).

(Contact your Cirrus Logic representative if you are attempting to build for a device for which the firmware file is not included in the device's fw folder, or if there is a mismatch between the firmware files used in the makefile and the fw folder.)

```
:
WARNING: The firmware file /c/Users/xxxxx/repo/mcu-
drivers_rel_4.18.0/cs47l35/fw/cs47l35_silkcoder_dsp2_010103.wmfw is missing. Please supply your
own firmware file. Contact Cirrus support for help.
WARNING: The firmware file /c/Users/xxxxx/repo/mcu-
drivers_rel_4.18.0/cs47l35/fw/SC_Voice_Marley_FB_SMSP_dsp3.wmfw is missing. Please supply your
own firmware file. Contact Cirrus support for help.
:
:
C:/Users/xxxxx/repo/mcu-drivers_rel_4.18.0/cs47l35/bsp/bsp_cs47l35.c:31:10: fatal error:
cs47l35_dsp2_fw_img.h: No such file or directory compilation terminated.
make: *** [makefile:152: /c/Users/xxxxx/repo/mcu-
drivers_rel_4.18.0/build/cs47l35/bsp/bsp_cs47l35.o] Error 1
```

3. The next step is to flash the .elf file onto the STM32. Once the executable is flashed and the STM32 is reset, the device will be configured to a minimal working state to be able to run through the use-cases. On a Cirrus Logic development system using the STM32 Nucleo board, the USER button is used to cycle the MCU Driver software through the use-case steps.

1.1 Prerequisites For Building

The MCU Driver SDK is developed and tested using a Windows host PC using a Bash-like terminal, the GCC compiler set suitable for the host machine, and an ARM cross-compiler suitable for the host machine. Alternatively, you may use a Linux command line terminal and correct compiler sets.

You will also need to have Python3 installed and available on the path, as the makefile invokes python tools scripts during the build, and a local install of Git.

Every OS will need the ARM cross compiler (arm-non-eabi-) which can be found at <https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads> (packages are available for Windows, Linux, and MacOS).

Additional recommendations to correctly build the SDK executable are enumerated below:

- On Windows host:
 - Bash-like terminal (Msys2).
 - GCC-Win64 compiler set as can be provided by MinGW64.
- Windows Subsystem for Linux (WSL) terminal:
 - Native GCC compiler set, normally available via the package manager.
 - ARM cross-compiler can be installed via the package manager (package "gcc-arm-none-eabi").

1.1.1 Code Layout Guide

For detailed folder layout and files for a particular device, refer to the appropriate Application Note for that device.

The MCU Driver source tree has the following folder structure. Only some of the notable files are described:

Table 1 MCU Driver Source Tree Structure

Folder/File	Relevant Folder or Files	Description
/	doc.zip release_notes.txt sdk_version.h LICENSE	Doxygen HTML-based source code documentation for the driver library source code. Release notes for the current and past releases of the MCU Driver Software Package. Macros for MCU Driver Software Package release version. Apache 2.0 License
/build	.elf, .map <device>/lib<device>.a	The ELF (executable) and MAP files created by the build process. Static library. a file will be created to link in with a separate MCU project.
/common	common.mk bsp_driver_if.h /bridge regmap.c/.h /compression /buffers	Files and modules common to all drivers, including a common makefile referenced by each device's own makefile. Defines the API that must be implemented to integrate the device driver into an MCU system. The bridge folder contains MCU code to handle requests from the Bridge API. Register map control port abstraction DSP Compressed buffer support
/common/platform_bsp	/eestm32int/platform_bsp.c /blackstar/platform_bsp.c ...	Implements the BSP-to-Driver code for the STM32 platform. Implements the BSP-to-Driver code for the Blackstar platform. ...
<device>	makefile <device>.c/.h <device>_ext.c/.h <device>_spec.h	The makefile for the device, makes use of common.mk The API and implementation of the device driver code. The API and implementation of the device driver code that is custom to the device. Definitions of types, arrays and constants from the device's datasheet as required by the driver.
<device>	/baremetal	Contains the main.c module for the 'baremetal' build.
<device>	/bsp	Contains the layer of code that will make calls to the driver API.
<device>	/config	Contains a WISCE® configuration script and any output from the wisce_script_converter tool.
<device>	/freertos	Contains the main.c module for the 'FreeRTOS' build.

Folder/File	Relevant Folder or Files	Description
<device>	/fw	Contains the firmware and tuning files if required for the application (contact your Cirrus Logic representative), and the output of the firmware_converter tool.
/third_party	/st /sbc	Location to add third party software, eg STM32CubeF4 (ST MCU HAL/Driver SDK for STM32F4-based MCUs). Bluetooth SBC library
/tools		Various useful tools and scripts to help with set up and configuration of the device. See Device Configuration Tools for details.

2 Integration Guide Overview

The main concern of a user of the MCU Driver SDK is to integrate the driver code into an MCU-based system's software environment.

It is assumed that configuring and generating the base code of the particular MCU software is the responsibility of the user and that it is in a working state. Physically connecting the Cirrus Logic device to the MCU system should have already happened and the required bus signals and other connections should have been fully tested and known to work.

This section only gives broad hints of the considerations that are needed to begin the integration steps by describing the important sub-systems. Review each section of this document and then refer to the individual Application Note for the specific device.

2.1 Driver to BSP Interface

The MCU vendor will normally provide a BSP layer of software or library which is the vendor API to perform hardware or other OS-related functions. Integrating the BSP layer to the driver is a major step for using the driver in a different MCU environment.

The bsp_driver_if_t structure (defined in common/bsp_driver_if.h) defines the API the integrator needs to implement to glue the Cirrus Logic driver to the MCU-based BSP layer. The structure is further detailed in a device's Application Note, but generally, it requires an I²C or SPI bus, GPIO pins, and interrupt operations to be implemented.

2.2 Device Configuration Data, Device Driver Instance

A higher-level application layer will create and populate instances of the driver structs and make calls to the driver API. In the SDK the code mimicking the application layer is in <device>/bsp/bsp_<device>.c.

The type <device>_t represents an instance of the driver and needs to be statically instantiated outside the scope of the driver source.

The structure <device>_bsp_config_t holds information needed to interface the driver to the BSP and needs to be statically defined and populated according to the development system's values. (See the [Important Data Structures](#) section for more details).

2.3 Baremetal and FreeRTOS Example Projects

Currently there are two projects (versions of the executable) that can be built for some of the devices - 'baremetal' and 'freertos'. The freertos version makes use of the STM32 CMSIS_OS operating system features which provides basic OS utilities such as threads, tasks, events, mutexes and other resource management objects. The freertos project may be more useful if you intend on handling asynchronous events in your project; some device features such as haptic control events may necessitate this approach.

There are additional build targets in the example project for devices that have an associated Cirrus Logic Demo board. Named demo boards include Blackstar, Holdout, and Live Oak.

2.4 Firmware Files

If a specific firmware is required for the device, then this will normally need to be integrated into the executable binary at compile time. Loading the firmware to the device during run time from external storage is also supported. The `firmware_converter` tool (in `/tools/firmware_converter`) can be used to convert firmware (.wmfw) and tuning (.bin) files into C modules that are data structures representing the firmware. These source files are placed in the `<device>/fw` folder and will be integrated into the executable during the build if named correctly. See the [Firmware Converter](#) section of the relevant Application Note for details.

2.5 WISCE Script Configuration

If an initial device configuration is required or provided in the form of a WISCE script, it can be integrated into the executable binary at build time. The tool `wisce_script_converter` (`/tools/wisce_script_converter`) takes in a WISCE script and generates a source and include file pair that define the C data table representing the device register writes in the WISCE script.

If the WISCE configuration script is named as `wisce_init.txt` and placed in the `<device>/config` folder, the makefile will invoke the `wisce_script_converter` tool automatically and generate the `<device>_syscfg_regs.c/.h` files, which are integrated into the executable.

See the [WISCE Script Converter](#) section of the relevant Application Note for details.

3 Overview of Cirrus Logic MCU Driver Development System

A bird's eye view of Cirrus Logic's internal development system for MCU Drivers is shown at [High Level Architectural Diagram](#). The device under development, in Cirrus Logic's mini-card format, sits on an Lochnagar2 development platform that is internal to Cirrus Logic. The LN2 has flexible pin routing and configuration and serves as the communication hub between external interfaces and the Cirrus Logic device by means of SPI/I²C/I²S buses and GPIO connections. The STM32 Nucleo board connects to the LN2 via an interconnect board (EESTM32INT); this routes the MCU's buses to the LN2. The STM32 has an on-board flash to receive the binary executable from a PC Host. The STLink goes to a dedicated STM32 on the Nucleo that acts as the debug host that then connects via JTAG to the target MCU on the Nucleo. The STM32 comes with the ST-Link firmware as default that sends and receives data from the virtual COM port on the Host.

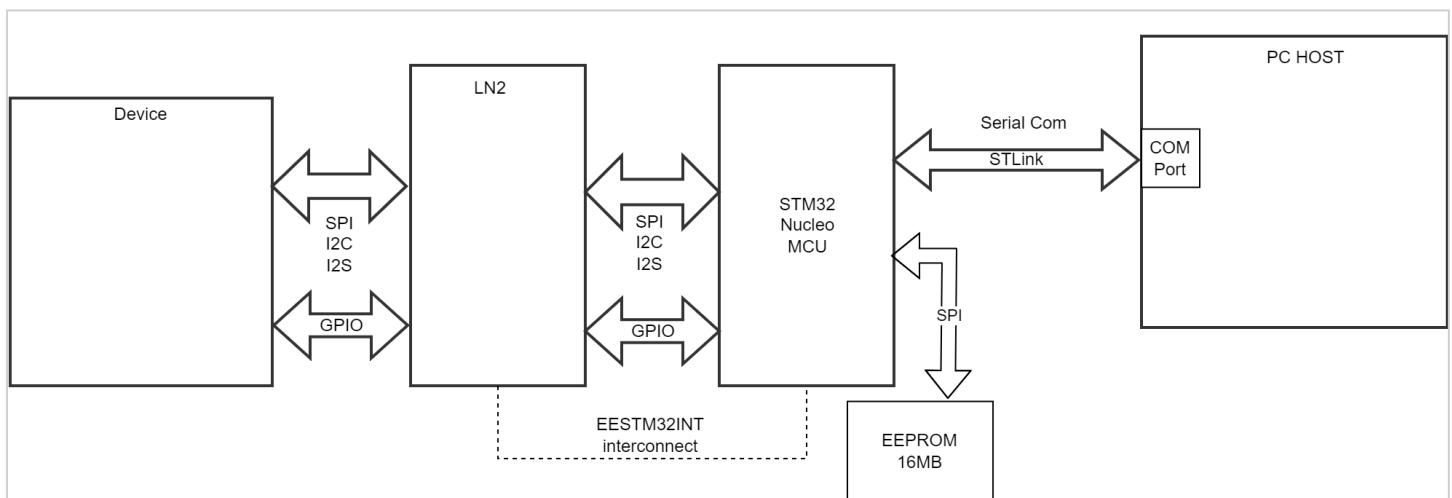


Figure 1 High Level Architectural Diagram

3.1 Detailed System Architecture

A more detailed diagram is given below which the reader can refer to as a reference when individual components are described.

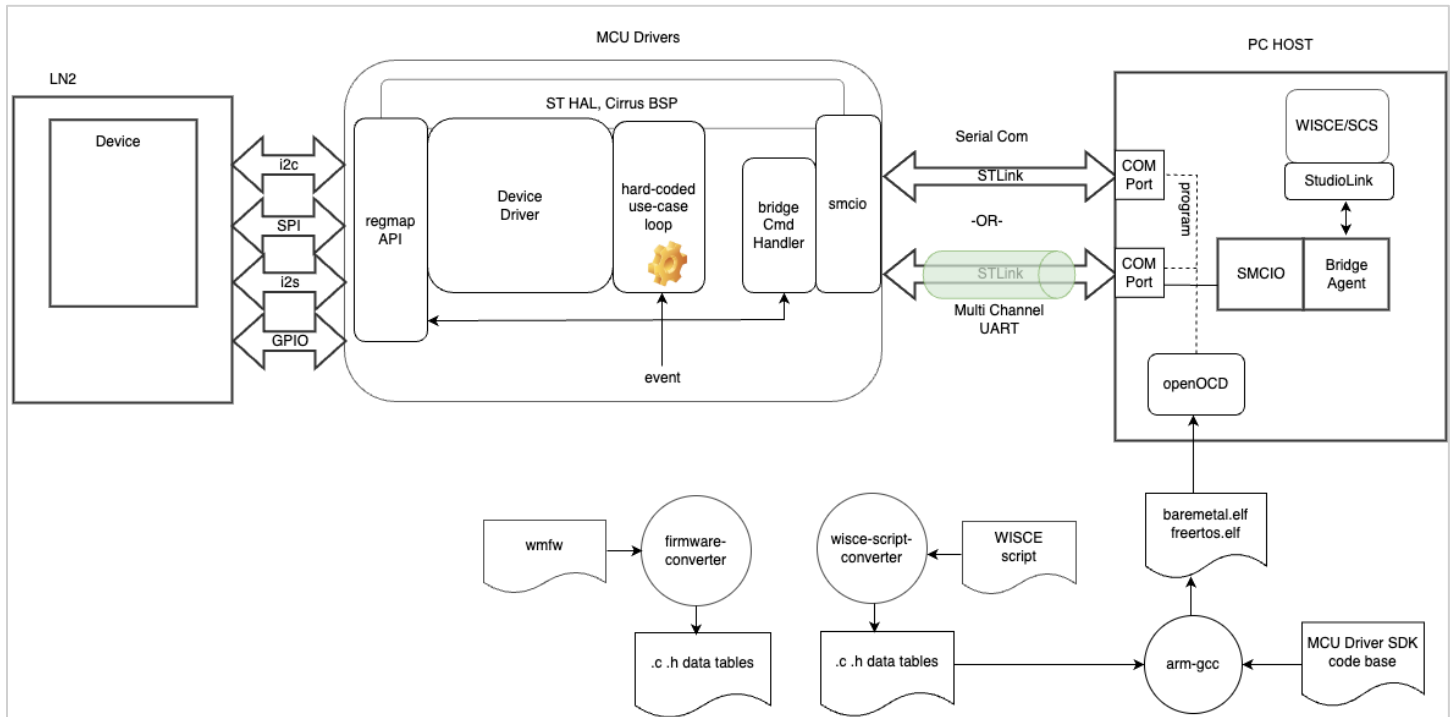


Figure 2 Detailed Diagram

4 Overview of Use-Case Code

This section details the application layer of the SDK which consists of the main-loop and use-case loops. The philosophy behind this code in the SDK is to offer examples of invoking the driver API to enable device features and achieve certain outcomes in a simple manner. The use-case loop relies on a button-press of the User button on the STM32 Nucleo board to step through the uses-cases in a round-robin manner.

The main function is located in either *baremetal/main.c* or *freertos/main.c*.

Step one is to initialize the device, the various bsp components and any other components requiring initialization, and ensure they are configured with any pre-configuration data that was built into the executable by use of the tools (wisce-script-converter and firmware converter, for example).

After initialization, main enters a while-loop which continually checks for the USER button press event and reacts with a state change to execute the next use-case code. The use-case example code attempts to execute some aspects the device is capable of in a simple manner. Each device has slightly different use-cases depending on its ability.

Generally speaking, the code in main makes generic calls to functions named as `bsp_xxxx` or `bsp_dut_xxxxx`; these are functions defined in the device's `bsp\bsp_<device>.c` file. The code in a `bsp\bsp_<device>` source file attempts to decouple the main code from the driver code of any specific device. The public API's in `bsp\bsp_<device>` call the actual device driver code. To do this the structure representing a driver instance (of type `<device>_t`) must be declared and is done so in `bsp\bsp_<device>`; this variable is required to be first initialized by main first calling `bsp_dut_initialize()`; it is then required to be passed to each driver API call.

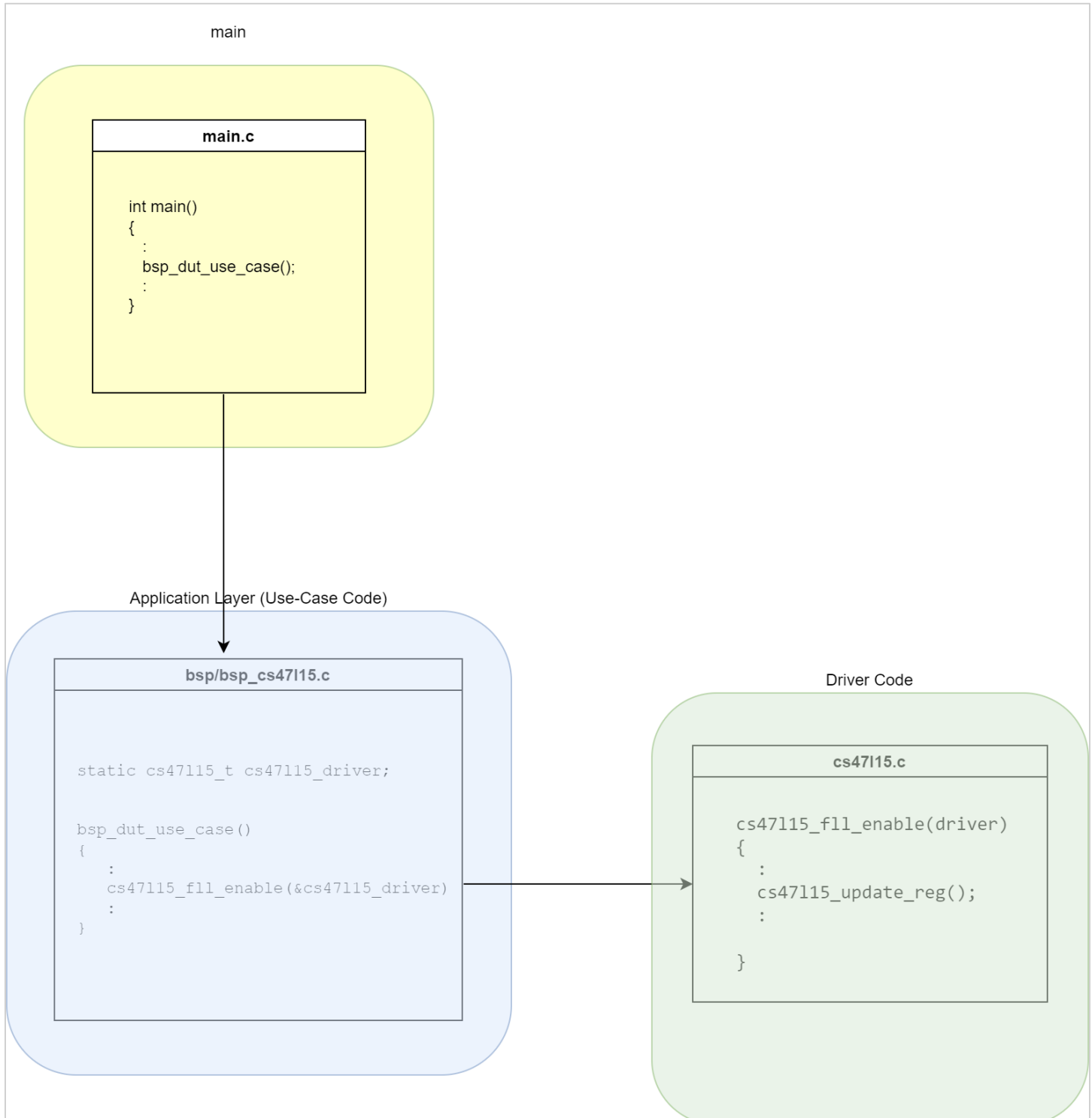


Figure 3 Main to Driver Flow

This mechanism makes it possible to have more than one driver being created and handled in a `\bsp\bsp_<devices>` source file if the set-up requires control of multiple Cirrus Logic devices (either the same type or entirely different) being present in the same platform.

5 Overview of Driver Code

The device driver modules are located at the top level under a folder named after the device part number. The modules usually consist of the core driver source and header, an extended API source and header, and device data specification source and header. For example:

- `cs40l26.c` / `cs40l26.h` – core device driver files
- `cs40l26_ext.c` / `cs40l26_ext.h` – extended driver API
- `cs40l26_spec.h` – definitions of constants, types and arrays from the device datasheet

Device driver code is mainly centered around implementing configuration and control of the device, monitoring the state the device is in, and responding to events. In most cases it does this by reading and writing to device registers via the regmap layer (`regmap.c`).

The driver code is designed to be independent of who calls the API and how it interfaces to an underlying platform. All of the public API's in the driver should require to be passed in the single instance of the device representation in the form of type `<device>_t`; and the code should keep this instance updated to reflect the state of the device.

It is up to the higher "application" layer (the use-case code found in `\bsp\bsp_<device>` source files) and in main to monitor application state and take action accordingly. And it is up to the regmap layer to abstract away the underlying interface to buses on the platform.

6 Important Data Structures

This section highlights some of the key data structures that may help in understanding how the driver and BSP code in the SDK hang together. Some of these areas in the BSP interface layer will need to be changed for either different values or different implementation entirely to integrate with a different platform. It is up to the integrator to decide whether to keep with the style of the example code or use an entirely different approach.

Most of the structures described below are contained in the single device driver structure instance `<device>_t`, for example, `cs40l50_t` as shown in the diagram below.

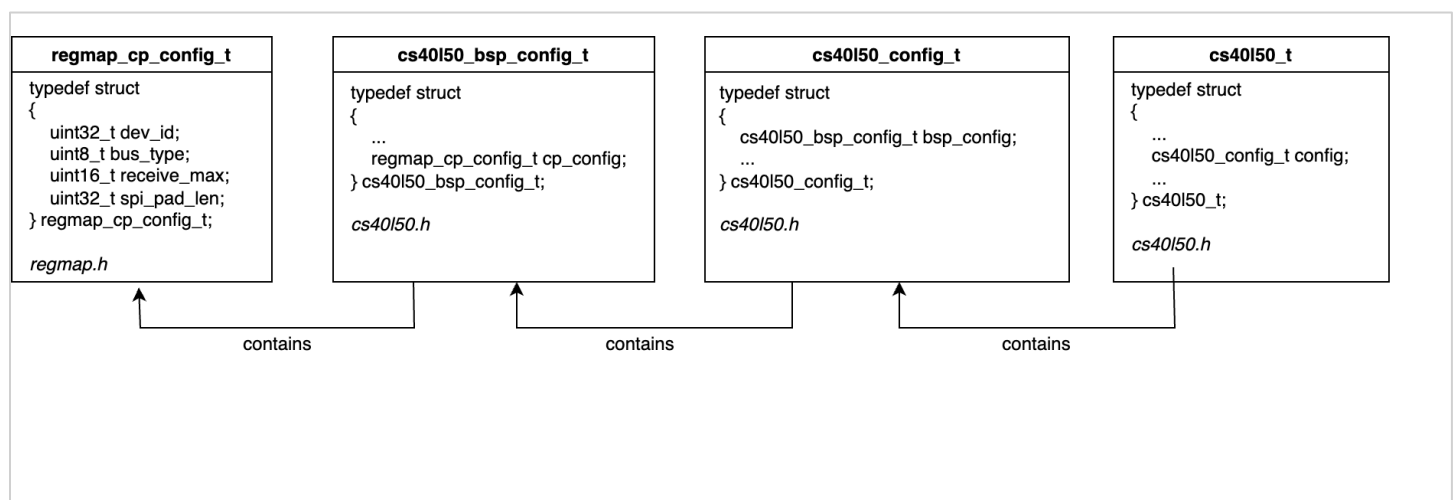


Figure 4 Example Diagram

6.1 regmap_cp_config_t

This structure defined in 'common/regmap.h' is the control port configuration for regmap API calls. Each device driver contains a correctly initialized instance of the regmap_cp_config_t structure.

The regmap (register map) enables various types of reads and writes to the device's array of control registers, which are ultimately carried out via the BSP over either an I²C or SPI bus. Of note here are the **bus_type**, **dev_id**, and **spi_pad_len** fields.

The bus_type is an enumeration, and their values are defined in common/regmap.h which reflects which bus this particular device is accessed on. Each device driver instance needs the correct bus_type enumeration to be defined in the regmap_cp_config_t structure. The enumeration is used mainly in regmap.c to help decide which BSP code to invoke based on the enumeration value.

```
common/regmap.h
Types of control port bus supported
```

```
#define REGMAP_BUS_TYPE_I2C           (0)
#define REGMAP_BUS_TYPE_SPI          (1)
#define REGMAP_BUS_TYPE_SPI_3000    (2)
#define REGMAP_BUS_TYPE_VIRTUAL      (3)
```

The dev_id is an enumeration, their values are defined in common/platform_bsp/platform_bsp.h.

```
common/platform_bsp/platform_bsp.h
```

```
#define BSP_DEV_ID_NULL              (0)
#define BSP_DUT_DEV_ID              (1)
#define BSP_LN2_DEV_ID              (2)
#define BSP_DUT_DEV_ID_SPI2         (3)
#define BSP_INTP_EXP_DEV_ID         (4)
#define BSP_EEPROM_DEV_ID           (5)
```

The dev_id enumeration is resolved in the BSP-to-driver layer, common/platform_bsp/leestm32int/platform_bsp.c for the STM32 platform for example, to identify which bus address the device is present on its particular bus type (the bus type is identified by the bus_type field). The platform_bsp.c code is then able to configure the correct GPIO pins in order to configure the bus and device address correctly. A different platform will have its own implementation for achieving this and is beyond the scope of this document.

The spi_pad_len field in the regmap_cp_config_t structure is the number of empty bytes needed to be placed between the initial register address and start of 32-bit data word during a SPI transaction. Refer to the silicon datasheet of the particular device to find what this value should be.

6.2 <device>_bsp_config_t

The <device>_bsp_config_t type is defined in the driver's include file (for example, cs47115.h) and statically declared in the application/use-case layer (for example, bsp/bsp_cs47115.c). This structure contains configuration values and callbacks required to interact with the BSP-Driver interface, as defined by the bsp_driver_if_g structure. When the driver uses the bsp_driver_if_t API, it will pass data from the <device>_bsp_config_t instance. The structure contains the regmap_cp_config_t instance to define bus related values, and other fields involved in driver to BSP interaction.

6.3 <device>_config_t

The <device>_config_t type is defined in the driver's include file (for example cs47115.h). The application/use-case layer (for example bsp/bsp_cs47115.c) will use an instance of this structure to represent each device. This structure references the register configuration data (syscfg_regs) required to initialize the device with, and also the instance of <device>_bsp_config_t (described above).

The register configuration data refers to an array of register address and value pairs which are required to initialize the

device to a base state. The configuration data is normally supplied as a hand-written WISCE script, and the data array pointed to by `syscfg_regs` is auto-generated at compile time (see <link-to-tools-section>).

6.4 <device>_t

The <device>_t type is defined in the driver's include file (for example, `cs47l15.h`) and statically declared in the application/use-case layer (for example, `bsp/bsp_cs47l15.c`). This structure represents the device driver throughout the code including configuration values and state by holding the various structures as described above and others that hold DSP and FLL configuration data. The <device>_t structure is passed into every API exposed by the driver.

6.5 bsp_driver_if_t

This is a structure of function pointers that define the BSP layer's public API. The BSP layer must implement each function according to the platform BSP. The driver relies on these functions to perform its register access, GPIO, IRQ, timer and callback features. Different platforms will likely have a different implementation of this API.

7 Overview of BSP Code

This section explains the interface of the driver to the BSP layer of code in the SDK (which is based on the STM32 Nucleo platform).

The main file which creates the BSP-to-driver interface is `platform_bsp.c`. This file will be unique for each hardware platform. In the SDK it is located under the `common/platform_bsp/<platform>` folder. Every different platform needs to implement this file and provide implementations for the API as defined by the type `bsp_driver_if_t`, in order to provide the low-level platform services required by the driver. There is a layer of abstraction between the driver and the platform-bsp called the regmap. The regmap layer abstracts register bus access operations required by the driver. This ensures that the driver code can be independent of the platform code.

The [BSP Layer Diagram](#) shows the layers involved when a call to a driver API is made from the higher "application" layer. In this case, the code for the CS47L15 device is used and the example shows how a typical call to the driver API `cs47l15_fll_enable` is made.

As shown, the application (the example use-case code in `bsp_cs47l15.c`) is required to enable the FLL on the device. It passes the driver instance `cs47l15_driver`, of type `cs47l15_t` (which was statically declared and initialized in the same module) to the driver API. This is vitally important as this structure holds the context of the device and the driver state, which gets used by lower layer code up to the BSP. Every driver API requires this instance to be passed to it.

When the driver code requires access to the control registers on the device it makes calls via the regmap layer (`common/regmap.c`) which in turn calls on the BSP-to-driver code in `platform_bsp.c`. The regmap (register map) code abstracts the various methods of register access required by the driver and other components of the SDK from the underlying bus access mechanisms that is ultimately performed by the platform BSP. The regmap API's require the `cp-config` (type `cs47l15_bsp_config_t`) passed to it from the driver's `cs47l15_driver` structure; this tells regmap which bus the device can be accessed on.

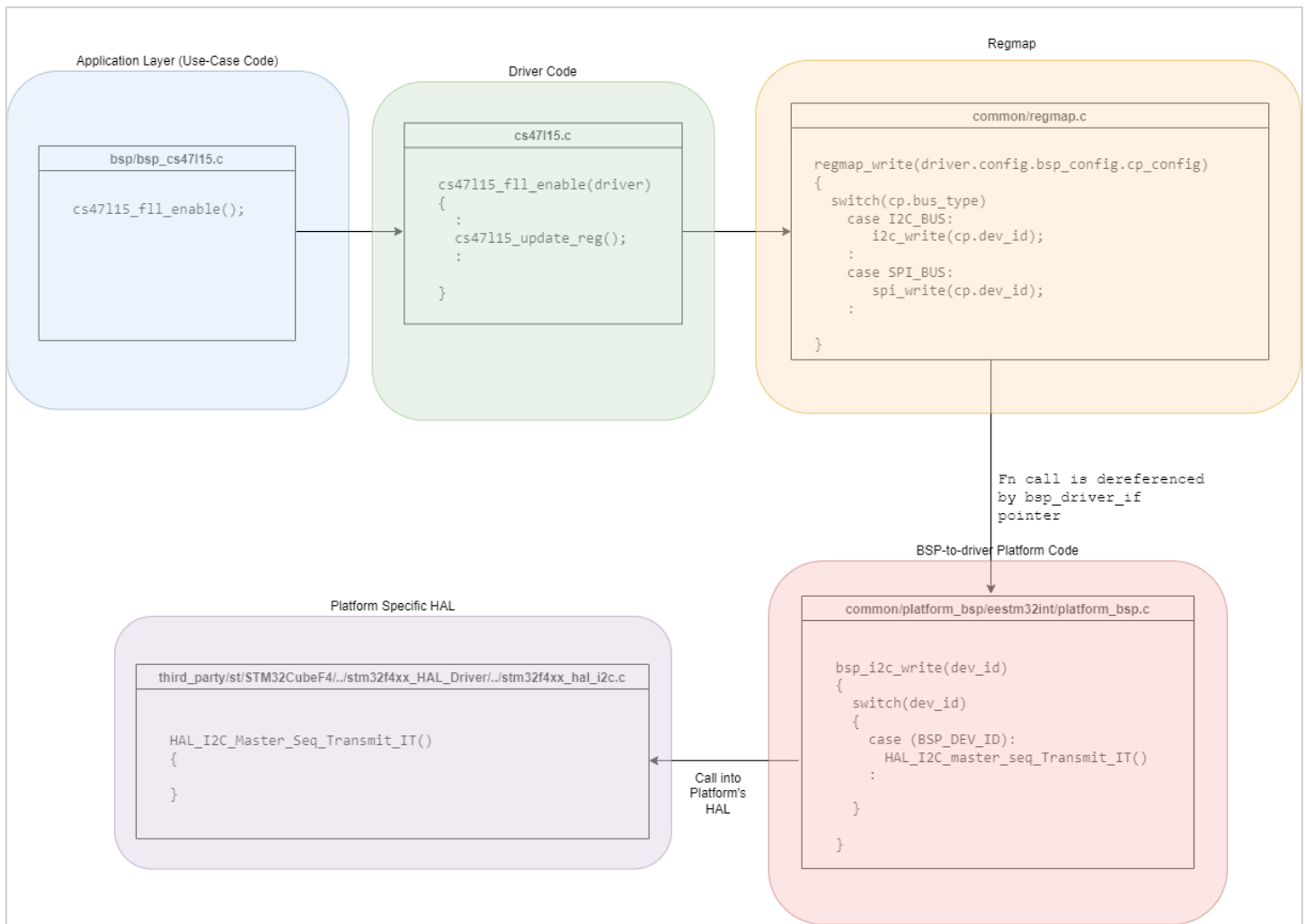


Figure 5 BSP Layer Diagram

8 Device Configuration Tools

The SDK contains a number of tools to help with configuring the device with initial register settings, load ADSP2 firmware data, and also to allow interfacing with the Cirrus Logic WISCE and SoundClear Studio® (SCS) desktop applications. The main tools are introduced below.

8.1 WISCE Script Converter

The WISCE Script Converter is a set of Python scripts that takes in initial device configuration data in the form of a "WISCE configuration script" and converts it into source data tables to be compiled into the executable at build time. The tool can also use outputs from SoundClear Studio®. The data gets loaded onto the device at start-up. The tool is located at `tools\wisce_script_converter`; the main file is `wisce_script_converter.py`.

```
% python3 tools/wisce_script_converter/wisce_script_converter.py -c c_array -p cs40150 -i cs40150/config/wisce_init.txt -o .
```

wisce_to_syscfg_reg_converter

Convert from either WISCE Script Text file or SCS generated CSV file to Alt-OS Syscfg Reg
SDK Version 4.19.0

Command: c_array

```
WISCE script path: cs40150/config/wisce_init.txt
Output path: .
Exported to:
./cs40150_syscfg_regs.h
./cs40150_syscfg_regs.c
```

8.2 Firmware Converter

The Firmware Converter is a set of Python scripts that takes in an ADSP2 firmware file (.wmfw) and/or tuning file (.bin) and converts them into source code data tables that get compiled into the executable at build time. The data gets loaded onto the device at start-up.

The tool is located at tools\firmware_converter. The main file is firmware_converter.py.

```
% python3 tools/firmware_converter/firmware_converter.py fw_img_v2 cs40150 ROM --wmdr
cs40150_wt.bin
```

```
firmware_converter
Convert from WMFW/WMDR ("BIN") Files to C Header/Source
SDK version 4.19.0
```

```
Command: fw_img_v2
Part Number: cs40150
WMFW Path: ROM
WMDR Path: cs40150_wt.bin
No suffix
Input Symbol ID Header: None
Generated Symbol ID Header cs40150_sym.h
Exported to files:
cs40150_fw_img.h
cs40150_fw_img.c
```

8.3 MCU Driver Bridge

The Bridge is a collective name given to enabling the MCU Driver development platform the ability to communicate with either the WISCE or SoundClear Studio (SCS) applications during run-time, to allow interacting with device registers in real-time. It is effectively an implementation of a "StudioBridge TCP/IP Connection" for MCU Drivers and is suited to run on the Cirrus Logic MCU Driver development system but can be adapted with changes to lower-level components for other set-ups if required (see below).

The purpose of the Bridge is to receive and process Bridge Protocol messages from clients such as WISCE and SCS such as register read and write operations to help with device configuration during run-time. A local installation of WISCE or SCS on the host is needed as a prerequisite.

As can be seen in the [Detailed Diagram](#) System Architecture diagram the Bridge consists of a Python script running on the host PC, the SMCIO component that runs on the host and on the MCU platform, and a bridge listener component on the MCU platform executable. The bridge listener component on the MCU is always running; the Python bridge script needs to be started manually at the command line in either a Windows command shell or bash-like terminal:

```
cd .\tools\bridge_agent\
python .\run_bridge.py -v -t 1
(type python .\run_bridge.py -h for help menu)
```

Not all devices supported by MCU Driver SDK have Bridge support enabled at the host; support can be added by the user and is described in the document "Device Integration to Bridge" in https://github.com/CirrusLogic/mcu-drivers/blob/master/common/bridge/Device_Integration_To_Bridge.md. This document also provides more details into the

Bridge components such as SMCIO.

As stated, the Bridge is geared to work with the Cirrus Logic MCU Driver development system in particular the serial connection between the MCU (STM32 board) and host which is handled by the SMCIO component. The Bridge makes it possible to adapt the the code (SMCIO component) for other configurations and this is also touched upon in the above document.

WISCE is now a legacy Cirrus Logic product; modern Cirrus Logic devices will likely only be supported by SCS.

9 Existing Material Links

Device App Note	Link	Description
CS47L63	TN0857R1 - CS47L63 MCU Driver User Guide	CS47L63 MCU Driver User Guide
CS40L26	Alt-OS - CS40L26 Support on Holdout	CS40L26 Support on "Holdout" evaluation board
CS40L50	AN0568R2 - CS40L50 MCU Driver Quick Start Guide	CS40L50 sample application bring-up guide on the "Blackstar" evaluation board
Tools App Note	Link	Description
Bridge	https://github.com/CirrusLogic/mcu-drivers/blob/master/common/bridge/Device_Integration_To_Bridge.md	Device Integration To Bridge

10 Revision History

Revision	Changes
R2 MAY 2026	• Removed confidential footer throughout the document.
R1 MAY 2024	• Initial release

Contacting Cirrus Logic Support

For all product questions and inquiries, contact a Cirrus Logic Sales Representative.

To find the one nearest you, go to www.cirrus.com.

IMPORTANT NOTICE

The products and services of Cirrus Logic International (UK) Limited; Cirrus Logic, Inc.; and other companies in the Cirrus Logic group (collectively either “Cirrus Logic” or “Cirrus”) are sold subject to Cirrus Logic’s terms and conditions of sale supplied at the time of order acknowledgment, including those pertaining to warranty, indemnification, and limitation of liability. Software is provided pursuant to applicable license terms. Cirrus Logic reserves the right to make changes to its products and specifications or to discontinue any product or service without notice. Customers should therefore obtain the latest version of relevant information from Cirrus Logic to verify that the information is current and complete. Testing and other quality control techniques are utilized to the extent Cirrus Logic deems necessary. Specific testing of all parameters of each device is not necessarily performed. In order to minimize risks associated with customer applications, the customer must use adequate design and operating safeguards to minimize inherent or procedural hazards. Cirrus Logic is not liable for applications assistance or customer product design. The customer is solely responsible for its overall product design, end-use applications, and system security, including the specific manner in which it uses Cirrus Logic components. Certain uses or product designs may require an intellectual property license from a third party. Features and operations described herein are for illustrative purposes only and do not constitute a suggestion or instruction to adopt a particular product design or a particular mode of operation for a Cirrus Logic component.

CIRRUS LOGIC PRODUCTS ARE NOT DESIGNED, TESTED, INTENDED OR WARRANTED FOR USE (1) WITH OR IN IMPLANTABLE PRODUCTS OR FDA/MHRA CLASS III (OR EQUIVALENT CLASSIFICATION) MEDICAL DEVICES, OR (2) IN ANY PRODUCTS, APPLICATIONS OR SYSTEMS, INCLUDING WITHOUT LIMITATION LIFE-CRITICAL MEDICAL EQUIPMENT OR SAFETY OR SECURITY EQUIPMENT, WHERE MALFUNCTION OF THE PRODUCT COULD CAUSE PERSONAL INJURY, DEATH, SEVERE PROPERTY DAMAGE OR SEVERE ENVIRONMENTAL HARM. INCLUSION OF CIRRUS LOGIC PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE FULLY AT THE CUSTOMER’S RISK AND CIRRUS LOGIC DISCLAIMS AND MAKES NO WARRANTY, EXPRESS, STATUTORY OR IMPLIED, INCLUDING THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR PARTICULAR PURPOSE, WITH REGARD TO ANY CIRRUS LOGIC PRODUCT THAT IS USED IN SUCH A MANNER. IF THE CUSTOMER OR CUSTOMER’S CUSTOMER USES OR PERMITS THE USE OF CIRRUS LOGIC PRODUCTS IN SUCH A MANNER, CUSTOMER AGREES, BY SUCH USE, TO FULLY INDEMNIFY CIRRUS LOGIC, ITS OFFICERS, DIRECTORS, EMPLOYEES, DISTRIBUTORS AND OTHER AGENTS FROM ANY AND ALL LIABILITY, INCLUDING ATTORNEYS’ FEES AND COSTS, THAT MAY RESULT FROM OR ARISE IN CONNECTION WITH THESE USES.

This document is the property of Cirrus Logic, and you may not use this document in connection with any legal analysis concerning Cirrus Logic products described herein. No license to any technology or intellectual property right of Cirrus Logic or any third party is granted herein, including but not limited to any patent right, copyright, mask work right, or other intellectual property rights. Any provision or publication of any third party’s products or services does not constitute Cirrus Logic’s approval, license, warranty or endorsement thereof. Cirrus Logic gives consent for copies to be made of the information contained herein only for use within your organization with respect to Cirrus Logic integrated circuits or other products of Cirrus Logic, and only if the reproduction is without alteration and is accompanied by all associated copyright, proprietary and other notices and conditions (including this notice). This consent does not extend to other copying such as copying for general distribution, advertising or promotional purposes, or for creating any work for resale. This document and its information is provided “AS IS” without warranty of any kind (express or implied). All statutory warranties and conditions are excluded to the fullest extent possible. No responsibility is assumed by Cirrus Logic for the use of information herein, including use of this information as the basis for manufacture or sale of any items, or for infringement of patents or other rights of third parties. Cirrus Logic, Cirrus, the Cirrus Logic logo design, and SoundClear are among the trademarks of Cirrus Logic. Other brand and product names may be trademarks or service marks of their respective owners.

Copyright © 2026 Cirrus Logic, Inc. and Cirrus Logic International Semiconductor Ltd. All rights reserved.

FreeRTOS and FreeRTOS.org are trademarks of Amazon Web Services, Inc.

I2C is a trademark of NXP B.V. CORPORATION NETHERLANDS.

SoundClear Studio and WISCE are registered trademarks of Cirrus Logic Inc.

STM32 is a registered and/or unregistered trademark of STMicroelectronics International NV or its affiliates in the EU and/or elsewhere. In particular, STM32 is registered in the US Patent and Trademark Office.